

PROBLEM STATEMENT

In urban environments, the increasing number of vehicles has made finding an available parking space a significant challenge. This leads to unnecessary circling of parking lots, resulting in traffic congestion, increased fuel consumption, and higher pollution levels. Traditional parking systems lack real-time availability information, causing driver frustration and inefficiency.

The challenge is to design and develop an embedded system that can:

- Automatically detect the occupancy status of multiple parking slots in real-time.
- Provide immediate, local feedback on slot availability within the parking area via an LCD display.
- Display a clear slot count summary (available/total) and individual slot statuses.
- Be cost-effective, scalable, and easy to implement using widely available components without requiring internet connectivity.
- Use the STM32 microcontroller for efficient and low-power embedded processing.

This project addresses these problems by creating a smart parking monitoring system using the STM32 Blue Pill board, IR sensors, an I2C LCD module, and an FTDI interface.

INTRODUCTION

Embedded systems play a vital role in modern automation and smart infrastructure. A Smart Car Parking System is a practical embedded application that enhances the efficiency of parking management without requiring cloud connectivity. This project, "Smart Car Parking System using STM32," leverages embedded technology to provide a real-time, on-site solution.

The system utilizes Infrared (IR) sensors placed in each parking bay to detect the presence of a vehicle. The STM32 Blue Pill board, known for its powerful ARM Cortex-M3 processor and rich peripheral support, acts as the central control unit. It reads data from all sensors, processes it to determine available slots, and displays a summary on a 16x2 LCD screen interfaced via an I2C module, which significantly reduces the wiring complexity.

An FTDI (Future Technology Devices International) module is used to bridge the USB-to-serial communication between the STM32 board and the host computer for programming and debugging purposes. On power-up, a startup animation sequence is displayed on the LCD, followed by continuous real-time monitoring of all parking slots.

This project serves as a foundational model for understanding sensor networks, I2C communication, and embedded programming on ARM-based microcontrollers, paving the way for more intelligent and connected parking infrastructures.

KEY COMPONENTS

1. STM32 Blue Pill Board (STM32F103C8T6)

- **Function:** Serves as the main microcontroller for reading IR sensors, driving the LCD display, and processing parking slot status.
- **Key Features:**
 - ARM Cortex-M3 32-bit processor running at 72 MHz
 - 72 MHz clock speed, 64KB Flash, 20KB SRAM.
 - Rich GPIO peripherals including 12C, SPI, USART and PWM.
- **Importance:** The brain of the project, handling all sensor reading and display output tasks efficiently.



2. IR Sensor Module (x2)

- **Function:** Detects the presence of a vehicle in a parking slot by emitting and receiving infrared light.
- **Key Characteristics:**
 - Outputs a digital signal (LOW when an obstacle is detected, HIGH when slot is empty).
 - Operating voltage: 3.3V - 5V.
- **Importance:** Acts as the primary input device to sense parking slot occupancy.



3. 16x2 LCD Display with I2C Module

- **Function:** Displays the real-time status of the parking system, including total available slots and individual slot status (EMP/FULL).
- **Key Features:**
 - 16 character x 2 line display.
 - I2C interface reduces wiring complexity.



- **Importance:** Provides local, on-site visual feedback for the system status.

4. Breadboard

- **Function:** A platform for prototyping and making temporary connections without soldering.
- **Importance:** Allows for easy circuit building, testing, and modification.



5. Jumper Wires

- **Function:** Used to establish electrical connections between components on the breadboard and the ESP32.
- **Types Used:** Male-to-Male (M-M) jumper wires, Male-to-Female (M-F) jumper wires.
- **Importance:** Essential for creating a flexible and reusable circuit prototype.



FTDI MODULE AND ITS INTERFACING

The FTDI (Future Technology Devices International) module is a USB-to-Serial converter that enables communication between a computer and microcontrollers like the STM32 Blue Pill. It translates USB data from the computer into UART (Universal Asynchronous Receiver-Transmitter) TTL-level signals that the STM32 can understand, and vice versa.

In this Smart Car Parking System, the FTDI module is used for two primary purposes: programming the STM32 Blue Pill board and monitoring serial output. Since the STM32 Blue Pill does not have a native USB bootloader by default, the FTDI module acts as the bridge for uploading the Arduino-IDE compiled code via the UART1 interface of the STM32.

The interfacing connections between the FTDI module and STM32 are as follows:

FTDI Module Pin	STM32 Blue Pill Pin
GND	GND
VCC (3.3V)	3.3V
TX	PA10 (RX1)
RX	PA9 (TX1)

To upload code, the STM32 BOOT0 pin is set HIGH (3.3V) to enter bootloader mode. After uploading via Arduino IDE with the STM32 board package installed, BOOT0 is returned to LOW for normal operation. The `Serial.begin(9600)` in code also uses the same UART via the FTDI to output sensor availability data for debugging.

CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

#define SENSOR1 PA0
#define SENSOR2 PA1

int totalSlots = 2;

void startupAnimation() {
    lcd.setCursor(0, 0);
    lcd.print(" SMART PARKING ");

    lcd.setCursor(0, 1);
    lcd.print(" Initializing ");

    for (int i = 0; i < 3; i++) {
        delay(500);
        lcd.print(".");
    }

    delay(1000);
    lcd.clear();
}

void setup() {
    Serial.begin(9600);
```

```

lcd.init();
lcd.backlight();

startupAnimation();

pinMode(SENSOR1, INPUT_PULLUP);
pinMode(SENSOR2, INPUT_PULLUP);
}

void loop() {
    int L1 = digitalRead(SENSOR1);
    int L2 = digitalRead(SENSOR2);

    int available = 0;

    if (L1 == 1) available++;
    if (L2 == 1) available++;

    // Line 1 → Slot Summary
    lcd.setCursor(0, 0);
    lcd.print("Slots:");
    lcd.print(available);
    lcd.print("/");
    lcd.print(totalSlots);
    lcd.print("  ");

    // Line 2 → Sensor status
    lcd.setCursor(0, 1);

```

```
if (L1 == 1)
    lcd.print("S1:EMP ");
else
    lcd.print("S1:FULL ");

if (L2 == 1)
    lcd.print("S2:EMP ");
else
    lcd.print("S2:FULL ");

Serial.print("Available: ");
Serial.println(available);

delay(500);
}
```


CODE EXPLANATION

This Arduino IDE program for the STM32 Blue Pill microcontroller is the core logic for the Smart Car Parking System. It handles IR sensor reading, LCD display output, and serial communication.

1. Library Inclusions

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>
```

- **Wire.h** – Handles I²C communication between STM32 and the LCD module.
- **LiquidCrystal_I2C.h** – Allows easy control of the LCD through the I²C interface.

2. LCD and Sensor Setup

```
LiquidCrystal_I2C lcd(0x27, 16, 2);

#define SENSOR1 PA0

#define SENSOR2 PA1

int totalSlots = 2;
```

- The LCD is initialized at I²C address 0x27 with 16 columns and 2 rows.
- IR Sensor 1 is connected to GPIO pin PA0 and Sensor 2 to PA1.
- totalSlots is set to 2, representing the total number of monitored parking spaces.

3. Startup Animation Function

```
void startupAnimation() { ... }
```

This function runs once at power-up to display a welcome sequence on the LCD. It prints "SMART PARKING" on line 1 and "Initializing..." on line 2, with dots appearing progressively. After a 1-second pause, the screen clears and the main monitoring loop begins.

4. Setup Function

```
void setup() { ... }
```

- **Serial.begin(9600)** – Initializes serial communication at 9600 baud for debug output via FTDI.
- **lcd.init() / lcd.backlight()** – Initializes the LCD and turns on the backlight.
- **startupAnimation()** – Calls the welcome animation function.
- **pinMode(SENSOR1/SENSOR2, INPUT_PULLUP)** – Configures sensor pins as inputs with internal pull-up resistors, ensuring stable readings.

5. Loop Function - Sensor Reading

```
int L1 = digitalRead(SENSOR1);
```

```
int L2 = digitalRead(SENSOR2);
```

Reads the digital logic level from each IR sensor every loop iteration:

- **HIGH (1):** No object detected → Slot Empty
- **LOW (0):** Object detected → Slot Full

6. Slot Count Display (LCD Line 1)

```
lcd.setCursor(0, 0);
```

```
lcd.print("Slots:"); lcd.print(available); lcd.print("/"); lcd.print(totalSlots);
```

The first line of the LCD displays a summary such as "Slots:2/2" or "Slots:1/2" based on the number of empty slots detected by the sensors.

7. Individual Slot Status (LCD Line 2)

```
if (L1 == 1) lcd.print("S1:EMP "); else lcd.print("S1:FULL ");
```

```
if (L2 == 1) lcd.print("S2:EMP "); else lcd.print("S2:FULL ");
```

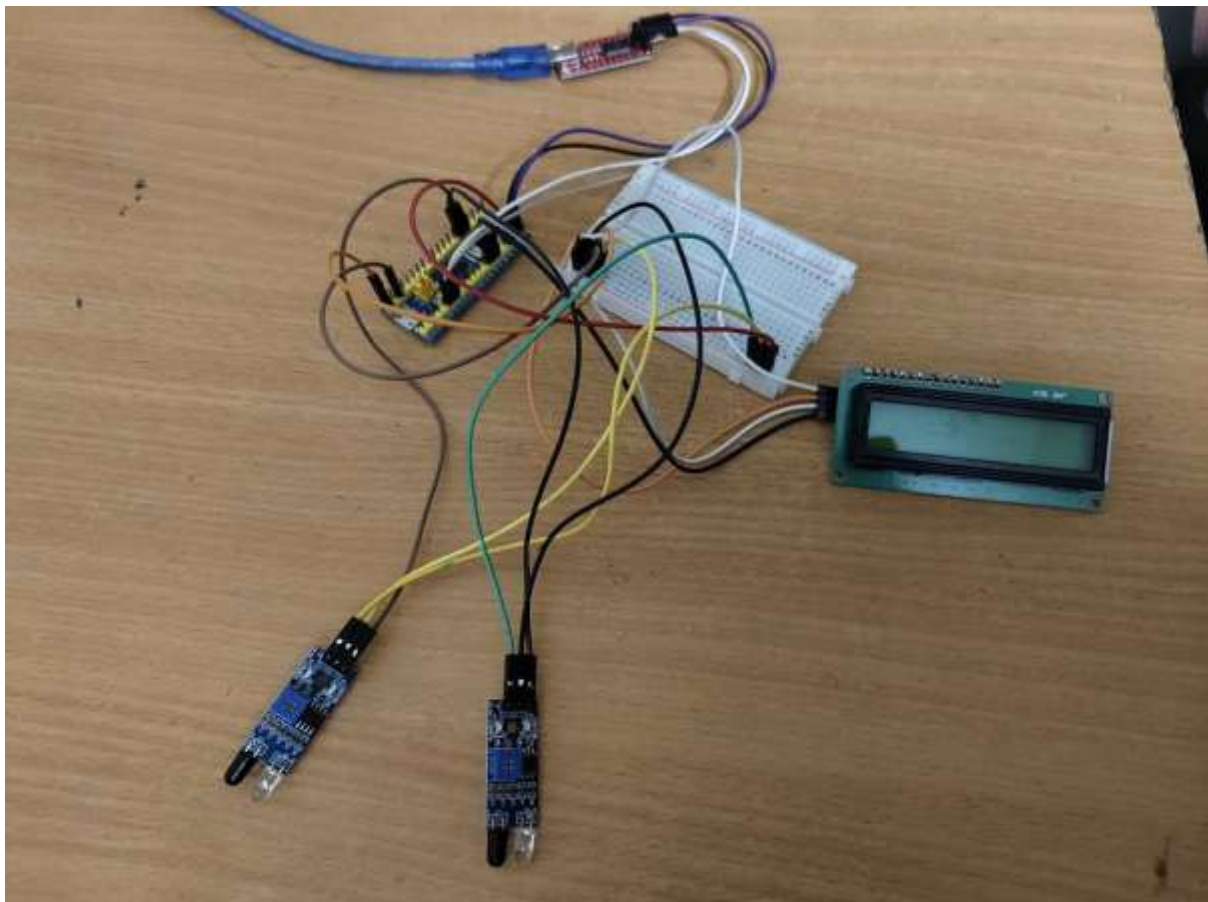
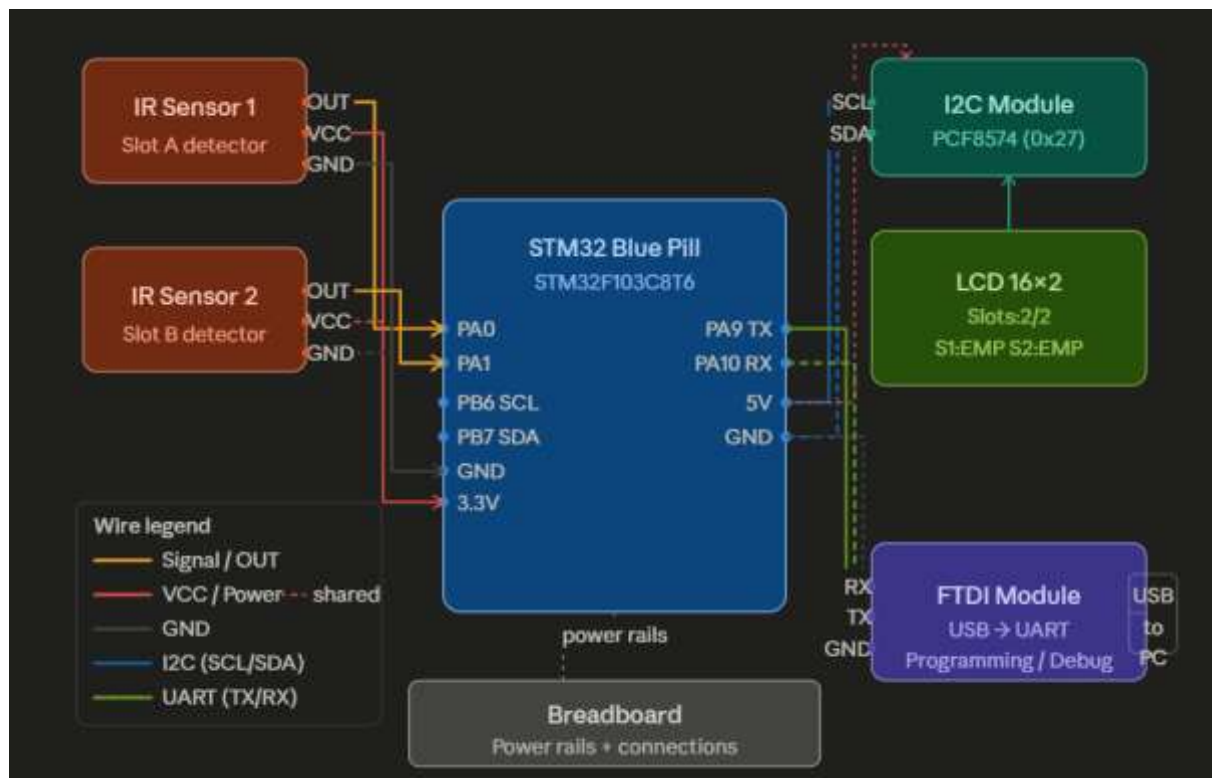
The second line displays each slot's individual status side by side. "S1:EMP" indicates Slot 1 is empty, while "S2:FULL" indicates Slot 2 is occupied. This gives users a quick glance at which specific slot is available.

8. Serial Output

```
Serial.print("Available: "); Serial.println(available);
```

The number of available slots is also printed to the serial monitor via the FTDI module, useful for debugging and verification during development.

CIRCUIT



RESULTS

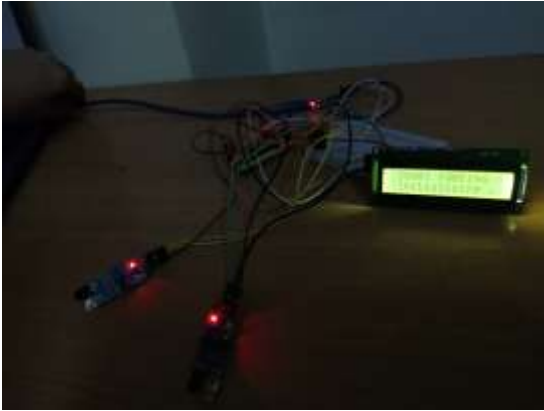


Figure 1: Welcome Message.

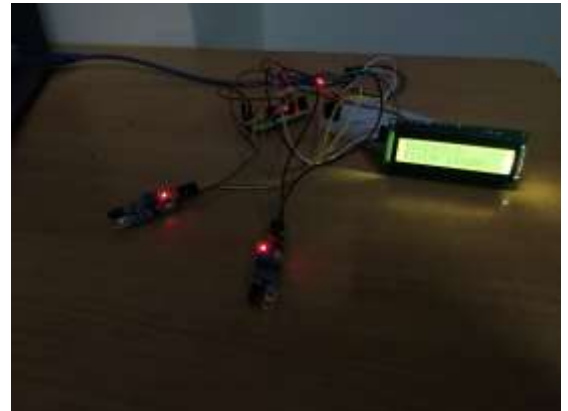


Figure 2: When parking is empty.

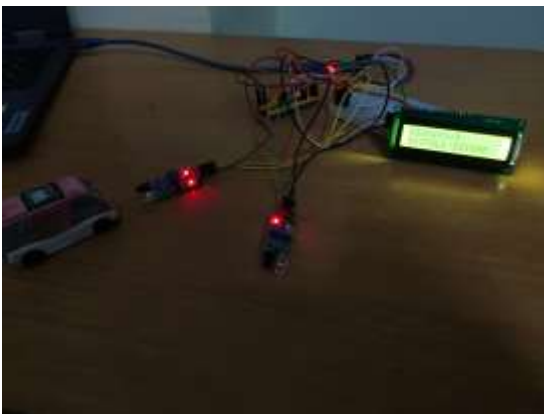


Figure 3: When Parking A is full and B is empty.



Figure 4: When Parking B is full and A is empty.



Figure 5: When both Parking A and B are full.

APPLICATIONS

The developed Smart Car Parking System has a wide range of practical applications in various domains:

1. **Shopping Malls and Multiplexes:** Helps direct customers to available spots, improving their experience and reducing traffic inside parking facilities.
2. **Airports and Railway Stations:** Manages the high volume of short-term and long-term parking efficiently, crucial for such high-traffic areas.
3. **Office Complexes and Corporate Campuses:** Allows employees and visitors to quickly find parking, saving time and reducing congestion during peak hours.
4. **Residential Apartments (Gated Communities):** Helps residents and their guests locate available parking slots within large apartment complexes.
5. **University and College Campuses:** Manages parking for students, faculty, and staff across large campus grounds.
6. **Hospitals:** Ensures that critical parking spaces, such as those for emergency vehicles or patient drop-off, are monitored and managed effectively.
7. **Public Event Venues:** Useful for managing parking during concerts, sports events, or fairs where parking demand is temporary and high.
8. **Standalone Embedded Deployments:** Since the system does not require internet, it can be deployed in remote or low-infrastructure areas where network connectivity is unavailable.

ADVANTAGES

The proposed system offers several significant advantages over traditional parking methods:

1. **Real-Time Information:** Provides live updates on parking slot availability, eliminating the need for drivers to physically inspect each spot.
2. **No Internet Dependency:** Unlike IoT-based systems, this system operates fully offline, making it reliable even in network-unavailable environments.
3. **Low Power Consumption:** The STM32 Blue Pill is an ARM Cortex-M3-based board known for its energy efficiency compared to WiFi-enabled boards like ESP32.
4. **Reduced Traffic Congestion:** By minimizing search time for parking, it directly contributes to reduced vehicular congestion in and around parking areas.
5. **Cost-Effective Solution:** Built using widely available, low-cost components like the STM32, IR sensors, and I2C LCD, making it an affordable automation solution.
6. **Scalability:** The system can be easily extended to monitor more slots by connecting additional IR sensors to the STM32's available GPIO pins.
7. **Clear Visual Feedback:** The I2C LCD provides both a slot count summary and individual slot status in a compact display format.
8. **Educational Value:** Serves as an excellent project for students to learn about embedded systems, I2C communication, and ARM microcontroller programming.

DISADVANTAGES

1. **Line-of-Sight Requirement of IR Sensors:** IR sensors can be affected by environmental factors like direct sunlight, dust, and rain, potentially leading to false readings.
2. **Limited Range:** The effective detection range of basic IR sensors is short, making them suitable only for close-range vehicle detection.
3. **No Remote Monitoring:** Unlike IoT-based systems with Blynk or MQTT, this system provides only local on-site LCD feedback with no mobile app or remote access.
4. **Limited Data Logging:** The basic implementation does not store historical data for analyzing parking patterns over time.
5. **Manual Code Upload:** The FTDI-based programming process requires manual BOOT0 pin toggling each time code needs to be updated, which is less convenient.
6. **Fixed Slot Count:** The number of monitored slots is defined at compile time; adding more slots requires code modification and recompilation.

FUTURE SCOPE

1. **IoT Integration:** Add an ESP8266 or SIM800L module alongside the STM32 to send parking data to an IoT platform like Blynk or ThingSpeak for remote monitoring.
2. **Advanced Sensors:** Replace IR sensors with ultrasonic sensors for better accuracy and range, or magnetic (reed) sensors for in-ground installation.
3. **LED Guidance System:** Install Green/Red LEDs above each parking spot to visually guide drivers to the exact location of a free space.
4. **Automatic Barrier Control:** Integrate a servo motor-controlled barrier that opens when a slot is available and closes when the lot is full.
5. **Multi-Floor Parking Support:** Expand the system using multiple STM32 units connected via RS485 or CAN bus for managing large multi-floor parking structures.
6. **Data Analytics Dashboard:** Create a web-based dashboard for parking lot operators to view usage statistics, peak hours, and capacity trends.
7. **Solar Power Integration:** Add a solar panel and battery management system to make the unit self-powered and suitable for outdoor deployment.

CONCLUSION

The "Smart Car Parking System using STM32" project was successfully designed, implemented, and tested. The system effectively demonstrates the integration of hardware components like the STM32 Blue Pill board, IR sensors, I2C LCD module, and FTDI interface to create a practical embedded solution for a real-world problem.

The prototype reliably monitors the status of two parking slots, displays total availability and individual slot status locally on the LCD, and outputs data to the serial monitor for debugging. The startup animation adds a polished touch to the user experience, while the efficient loop structure ensures timely updates every 500 milliseconds.

While the current system is a standalone embedded prototype, it provides a solid foundation for further development. The knowledge gained in sensor interfacing, I2C communication, and STM32 programming is invaluable. The identified future scopes, such as IoT integration, LED guidance, and solar power, present exciting pathways for further development.

In conclusion, this project not only achieves its technical objectives but also highlights the capability of ARM-based microcontrollers in building embedded smart systems. It serves as a comprehensive learning experience and a stepping stone towards more advanced industrial-grade parking management solutions.

Course Outcomes:

CO 4: Ability to design and implement embedded solutions for real-world problems.

CO 5: Develop skills in interfacing sensors and actuators with microcontrollers like STM32.

REFERENCES

- **STMicroelectronics.** (2023). STM32F103C8T6 Datasheet. Retrieved from <https://www.st.com/resource/en/datasheet/stm32f103c8.pdf>
- **Arduino.** (n.d.). Arduino Reference. Retrieved from <https://www.arduino.cc/reference/en/>
- **Roger Clark.** (2023). STM32duino (Arduino STM32) Documentation. Retrieved from https://github.com/rogerclarkmelbourne/Arduino_STM32
- **FTDI.** (2023). FT232R USB UART IC Datasheet. Retrieved from <https://ftdichip.com/products/ft232r/>
- **Adafruit.** (n.d.). IR Sensor Overview. Retrieved from <https://learn.adafruit.com/ir-sensor>
- **Arduino Blog.** (2023). Getting Started with the LCD 1602 Display. Retrieved from <https://blog.arduino.cc/2020/06/22/getting-started-with-the-lcd-1602-display/>
- **TutorialsPoint.** (2023). Embedded Systems Tutorial. Retrieved from https://www.tutorialspoint.com/embedded_systems/index.htm